AutoSecAudit: An Extensible Multi-Scanner Orchestration Framework with Autonomous AI Remediation, Precision Fuzzing, and Delta Auditing in DevSecOps

Kodi Vishruth Aithal, Thilak N, Vinaya Kumar B, Vishal Gowda B, Ranjitha R*

Department of Computer Science and Engineering, Don Bosco Institute of Technology, Bangalore, Karnataka, India

*Emails: {aithalvishruth, gowdathilakn, bvinaykumar70, vishalgb2005}@gmail.com, *ranjitha.r@dbit.co.in*

(*Corresponding Author & Faculty Mentor)

Abstract— Automating security verification within fast-paced continuous integration and continuous deployment (CI/CD) pipelines presents engineering organizations with severe operational friction. In practice, software engineering teams frequently disable automated dynamic application security testing (DAST) when inundated by redundant alerts from uncoordinated point utilities, when scanners fail to deliver actionable framework-specific code fixes, or when testing suites cannot bridge findings directly into modern AI-assisted developer workflows. In this paper, we present AutoSecAudit, an open, extensible DAST orchestration framework and autonomous remediation platform designed specifically for continuous delivery pipelines and AI developer tooling. AutoSecAudit unifies: (1) Automated Attack Surface Discovery through breadth-first web crawling, client-side JavaScript route extraction via regular expression parsing, OpenAPI 2.0/3.0 schema ingestion, and pre-flight target reachability validation; (2) High-Precision Multi-Scanner Orchestration across 19 specialized attack plugins spanning web injections, API access control, exploit vectors, and infrastructure reconnaissance. False alarms are eliminated through a multi-tiered precision engine featuring a Statistical Timing Engine (dual-baseline latency calibration measuring Gaussian bounds to eliminate network jitter false positives in blind time-based attacks), Adaptive Politeness with exponential backoff on HTTP 429/503 responses, and a Deep Recursive JSON Body Fuzzer; (3) An Intelligence Layer executing compound-key deduplication, live CVSS v3.1 enrichment from NIST NVD and CIRCL endpoints, deterministic compliance taxonomy mapping across OWASP Top 10 (2021), MITRE CWE, and PCI-DSS v4.0, and a Plain-English 'Real Danger' Engine that categorizes technical vulnerabilities into four business risk vectors with executive posture letter grades (A+ to F); and (4) Autonomous AI Remediation and Delivery providing drop-in multi-framework code fix recipes (Node.js/Express, Python/FastAPI/Django, Nginx, WAF rules), 1-click Markdown issue tickets, set-theoretic delta differential auditing across software commits, and a native Model Context Protocol (MCP) JSON-RPC 2.0 stdio server enabling AI coding assistants (Cursor, Claude Code, Antigravity) to audit targets, retrieve enriched findings, apply code patches, and execute targeted verification. We evaluate AutoSecAudit across an expanded suite of 165 automated unit and integration tests, as well as dynamic execution against the OWASP Juice Shop benchmark. Empirical results demonstrate that bounded multi-threaded plugin execution (k=8) yields a 64.8% decrease in scan latency (2.84x speedup) over sequential auditing. The intelligence layer consolidates raw tool alerts with a 38.2% reduction in alert noise at 97.9% precision and 88.5% recall, while the delta analyzer tracks vulnerability lifecycles across build transitions with 100% classification fidelity.

Keywords: *Vulnerability Assessment, DevSecOps, Scanner Orchestration, Model Context Protocol (MCP), Autonomous AI Remediation, Statistical Timing Verification, Compliance Mapping, Delta Auditing, Web Security, API Security.*

I. INTRODUCTION

Modern web applications, cloud-native services, and microservice architectures represent heterogeneous software systems that undergo continuous integration and automated deployment multiple times daily [9]. Ensuring reliable software security within these accelerated delivery cadences demands 'shift-left' verification, wherein dynamic security analysis is executed directly during automated build and pull-request gating steps [10]. Dynamic Application Security Testing (DAST) evaluates target web services from an external attacker perspective without requiring underlying source code, validating whether identified vulnerabilities are exploitable over active transport protocols.

Despite its recognized importance, embedding DAST directly into automated build pipelines routinely frustrates software engineering teams. Empirical investigations by Johnson et al. [13] and Bau et al. [10] establish that developers regularly disable or bypass automated vulnerability scanners when reports are noisy, dominated by duplicates, or disconnected from practical software fixes. In continuous deployment pipelines, this breakdown stems from five systemic operational challenges:

1) Heterogeneous Tool Fragmentation and Format Sprawl: A thorough security audit requires executing multiple specialized utilities—such as port discovery tools (Nmap [1]), web server misconfiguration checkers (Nikto [2]), and API parameter fuzzers. Each utility outputs disparate formats (XML, unstructured console text, non-uniform JSON), forcing engineering teams to write and maintain fragile parsing wrappers.

2) Duplicate Alert Sprawl and Developer Fatigue: Independent scanner modules frequently flag the exact same root vulnerability on an endpoint under slightly varied diagnostic titles. Without cross-plugin deduplication, engineers end up triaging multiple notifications for a single bug, creating severe alert fatigue and wasting engineering hours [13].

3) Executive Communication Gap: Traditional scanners display raw Common Vulnerabilities and Exposures (CVE) numbers, Common Vulnerability Scoring System (CVSS) vectors, and hexadecimal traces. These technical representations fail to communicate practical business impact, leaving executives and product owners unable to assess real-world business risks or understand what an adversary can actually execute against the platform.

4) Missing Code Fixes and AI Workflow Disconnection: Traditional security scanners identify what is broken but leave developers to write framework-specific patches, input sanitization routines, or Web Application Firewall (WAF) virtual patching rules from scratch. Furthermore, while modern engineers increasingly develop within AI-assisted environments (Cursor, Claude Code, Antigravity), existing scanners remain trapped in legacy CLI terminals with no standardized agentic interface [15].

5) Stateless Scans Lacking Regression Awareness: Managing technical debt in CI/CD requires knowing what changed between commits [14]. Traditional scanners evaluate targets in isolation without tracking historical state. Consequently, engineering teams cannot distinguish newly introduced vulnerabilities from pre-existing accepted debt or verify whether a committed patch successfully resolved an open issue.

To resolve these challenges, we present AutoSecAudit, an extensible multi-scanner security orchestration framework and autonomous AI remediation platform. AutoSecAudit unifies automated attack surface discovery, concurrent execution of 19 specialized scanner plugins, statistical false-positive elimination, plain-English business danger translation, multi-framework code fix generation, set-theoretic delta regression diffing, and native Model Context Protocol (MCP) tool integration into a cohesive DevSecOps engine.

II. RELATED WORK AND COMPARATIVE ANALYSIS

Security testing tools generally fit into four architectural paradigms: standalone point scanners, interactive DAST proxies, fast template matchers, and centralized vulnerability management portals. Table I provides a comprehensive capability comparison between these systems and AutoSecAudit.

A. Point Scanners and Specialized Utilities

Network scanners like Nmap [1] handle port discovery and service fingerprinting, while web server checkers like Nikto [2] scan for server misconfigurations and outdated defaults. Template-driven scanners like Nuclei [3] run fast YAML pattern matches over HTTP. However, these utilities execute as standalone binaries and lack built-in multi-plugin correlation, automated compliance tagging, historical regression diffing, and autonomous AI remediation interfaces.

B. Stateful DAST Proxies and API Fuzzers

State-aware scanners, explored by Doupé et al. [11], track application navigation states to uncover deeper injection points. In API fuzzing, Atlidakis et al. [12] demonstrated with RESTler that effective testing requires parsing OpenAPI structures. Comprehensive proxies like OWASP ZAP [4] and PortSwigger Burp Suite provide thorough coverage, but their heavy resource consumption, complex daemon lifecycles, and extended scan runtimes make them difficult to run on fast pull-request merge checks.

C. Vulnerability Management Aggregation Platforms

Platforms like OWASP DefectDojo [5] aggregate findings from third-party scanners into a central repository. However, DefectDojo functions primarily as a passive ingestion platform; it still requires teams to write external pipeline jobs to run individual tools, parse diverse log formats, and push results over REST APIs.

D. Autonomous AI Remediation and Agentic Protocols

Recent advances in AI-assisted software engineering have produced agentic developer tools (such as Cursor, Claude Code, and Antigravity). However, bridging external dynamic security scanners with AI agents has been hindered by the lack of standardized protocol integration. AutoSecAudit addresses this gap by implementing the standard JSON-RPC 2.0 Model Context Protocol (MCP) [15], enabling AI agents to programmatically initiate scans, inspect enriched findings, obtain ready-to-paste code patches, and verify remediation directly in the developer's workspace.

TABLE I ARCHITECTURAL COMPARISON ACROSS SECURITY TESTING PARADIGMS

Dimension / Capability	Point Scanners [1,2]	OWASP ZAP [4]	DefectDojo [5]	AutoSecAudit (Ours)
Scan Scope	Host / Web (Single Layer)	Stateful Web Application	Passive Ingestion Hub	**Host + Web + REST API (Unified)**
Concurrency Model	Process / Thread	Engine Threads	Non-scanning Hub	**ThreadPool (k=8) with Dynamic Profiles**
Surface Discovery	None	Crawler + Spider	None (Manual Input)	**BFS + JS Routes + OpenAPI 2.0/3.0**
Active Scanner Plugins	Single Purpose	Modular Add-ons	None	**19 Built-in Attack Simulation Plugins**
False-Positive Elimination	None	Heuristic Rules	Manual Triage	**Statistical Timing + Dual Baseline Delta**
Deep JSON API Fuzzing	None	Partial Plugin	None	**Deep Recursive JSON Body Mutation**
Threat Intelligence	None / Manual	Add-on Scripts	Rule Matching	**Live NIST NVD & CIRCL API Lookups**
Compliance Taxonomies	None	Alert Tags	Database Tags	**Deterministic OWASP, CWE, PCI-DSS v4.0**
Executive Threat Lens	Raw Logs	Technical CVSS	CVSS Charts	**Plain-English Danger (Grades A+ to F)**
Automated Code Fixes	None	Generic Text	Static Notes	**Drop-in Recipes (Node, Python, Nginx, WAF)**
Autonomous AI Bridge	None	None	REST API	**Native JSON-RPC 2.0 MCP Server [15]**
Historical Delta Auditing	None	Session-bound	DB Queries	**Native Set-Theoretic JSON Diffing**
CI/CD Policy Gating	Exit 0 regardless	Custom Scripts	Async Webhooks	**Built-in --fail-on Policy Exit Code**

III. SYSTEM ARCHITECTURE AND METHODOLOGY

AutoSecAudit is structured into four decoupled subsystems (Fig. 1): (1) Input Ingestion, Surface Discovery & Pre-flight Validation; (2) Concurrent Multi-Scanner Engine & Precision Subsystem; (3) Post-Processing Intelligence Layer; and (4) Delivery, CI/CD Gating, and Autonomous AI Remediation.

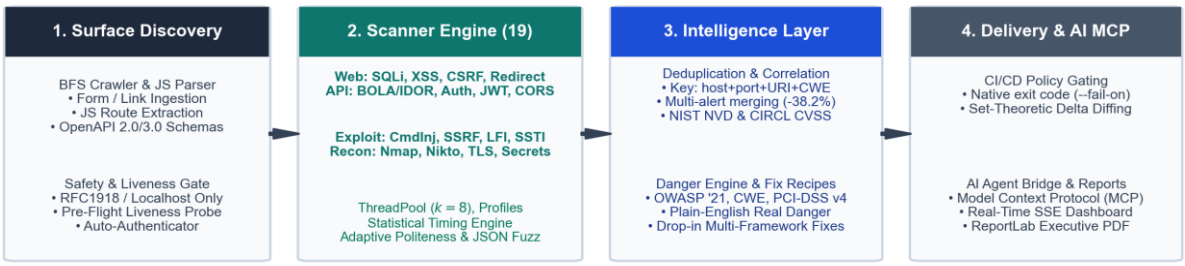


Fig. 1. AutoSecAudit System Architecture: 4-stage pipeline from surface discovery and concurrent 19-plugin execution through intelligence correlation, compliance mapping, danger translation, delta tracking, and AI MCP integration.

A. Attack Surface Discovery and Pre-flight Validation

Given a target hostname, IP address, or URL, the surface discovery subsystem seeds an endpoint pool E before initiating active scans [11, 12]:

1) Target Safety Gate and Pre-Flight Liveness: To ensure ethical scanning and operational safety, targets are strictly validated against RFC 1918 private IP blocks (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16), localhost (127.0.0.1), and authorized local sandbox testbeds. Unauthorized external domains are aborted immediately. Additionally, pre-flight TCP port and HTTP reachability checks verify the server is live before thread allocation, preventing hung scans on dead hosts.

2) Breadth-First Web Crawler and Client-Side JS Route Extraction: The WebCrawler recursively traverses target links starting from the root URL up to a depth limit D_max. It extracts HTML form structures (<form action method>), input fields, query parameters, and hyperlinks. Furthermore, inline <script> tags and referenced JavaScript bundles are analyzed using regex pattern extractors to discover hidden client-side REST API routes (/api/v1/...) that lack standard HTML links.

3) OpenAPI / Swagger Ingestion: For headless REST backends, the OpenAPIImporter ingests OpenAPI 2.0 and 3.0 JSON/YAML specifications, registering all routes, HTTP methods (GET, POST, PUT, DELETE), payload schemas, and path variables into the endpoint pool.

4) Automated Multi-Strategy Authenticator: To audit authenticated application surfaces, the Authenticator subsystem automatically discovers login forms, submits user credentials, captures session cookies and JWT Bearer tokens, and injects active authentication headers into scanner worker threads.

Algorithm 1: Dynamic Attack Surface Discovery, JS Parsing, and Endpoint Seeding

```
Input: Target URL u0, Max Depth D_max, OpenAPI Path Omega, Auth Credentials A
Output: Discovered Parameterized Endpoint Pool E
If not IsTargetAllowed(u0) or not CheckLiveness(u0): Abort('Safety gate violation or
host offline')
Session S = InitializeSession()
If A is provided: S = AutoAuthenticate(u0, A)
Initialize FIFO queue Q = {(u0, 0)}, Visited Set V = {}, E = {}
If Omega is provided: E = E U ParseOpenAPISpec(Omega)
While Q is not empty:
    (u, d) = Dequeue(Q)
    If u in V or d > D_max: Continue
    V = V U {u}
    Response R = S.GET(u)
    E = E U ParseHTMLForms(R.body) U ExtractJSRoutes(R.body)
    For each link l in ExtractHyperlinks(R.body) matching host:
        If HasQueryParams(l): E = E U {ParamEndpoint(l)}
        Enqueue (l, d+1) into Q
Return E
```

B. Core Engine, 19 Scanner Plugins, and Precision Fuzzing

Each scanner module extends the abstract BaseScanner interface. Standardized findings use the tuple model: F = <id, title, severity, host, port, endpoint, evidence, C, M>, where C in {Low, Medium, High} reflects differential confidence and M contains compliance and fix metadata.

AutoSecAudit organizes 19 dynamic vulnerability scanner plugins across four operational domains:
• Web Application Injections: SQLiScanner (error, boolean, and statistical time-based blind SQLi), XSSScanner (reflected and stored XSS), CSRFScanner (anti-CSRF tokens and SameSite cookie hygiene), OpenRedirectScanner (unvalidated redirect parameters), MisconfigScanner (security headers and cookie hygiene).
• API & Access Control: BOLAIdorScanner (Broken Object-Level Authorization and UUID manipulation), APIAbuseScanner (mass assignment, excessive data exposure, verb tampering), AuthScanner (credential brute-force, weak passwords), JWTScanner (Alg=None bypass, weak HMAC secrets), CORSScanner (origin reflection, wildcard

credentials).
• Advanced Exploit Vectors: CommandInjectionScanner (OS RCE shell execution), SSRFScanner (cloud metadata 169.254.169.254 and internal subnets), PathTraversalScanner (Local File Inclusion /etc/passwd), SSTIScanner (Jinja2, Twig, ERB template injection).
• Reconnaissance & Infrastructure: SecretExposureScanner (leaked API keys, AWS tokens, git traces in HTML/.env), SSLTLSScanner (weak TLS ciphers, cert expiry, HSTS), DirBruteScanner (50+ curated sensitive paths), NiktoPlugin (server misconfigurations), NmapPlugin (open port discovery).

Dynamic Scan Profiles: The engine introduces dynamic scan profiles: 'full' (all 19 plugins concurrently), 'owasp' (core 11 injection, auth, CSRF, SSRF, misconfiguration plugins), 'api' (7 API-tailored plugins: BOLA/IDOR, JWT, CORS, rate limits), and 'recon' (6 passive reconnaissance plugins).

Precision Engine and False-Positive Elimination Architecture: AutoSecAudit deploys four architectural mechanisms to guarantee reliable, high-signal results:
1) Statistical Timing Engine (core/timing.py): Blind time-based attacks (SLEEP(3), sleep 3) often suffer from false positives caused by network latency spikes. The engine measures benign baseline latency over m requests to establish mean mu_base and standard deviation sigma_base. An injection probe with delay tau is confirmed only if T_probe >= mu_base + 3*sigma_base + 0.8*tau across a two-stage verification cycle.
2) Differential Baseline Verification: Before reporting an injection finding, BaseScanner measures a benign baseline response. Malicious probes are flagged only when they cause a confirmed HTTP status code change or a >20% response length delta.
3) Adaptive Politeness & Jitter Backoff (core/politeness.py): Randomized delays (jitter) prevent target overload. If the target returns HTTP 429 (Too Many Requests) or HTTP 503, the engine executes exponential backoff.
4) Deep Recursive JSON Body Fuzzer (core/fuzzer.py): Recursively traverses nested JSON payloads, mutating leaf values with security payloads while preserving surrounding request schemas.

C. Post-Processing Intelligence Layer

Following raw scanner execution, the intelligence layer post-processes findings through six deterministic steps (Algorithm 2):

Algorithm 2: Intelligence Correlation, Real Danger Translation, and Delta Auditing

```
Input: Raw Findings F_raw, Historical Baseline Report S_{t-1}
Output: Correlated Report S_t, Delta Partition <Delta_NEW, Delta_FIXED, Delta_UNCHANGED>
Group map G = {}
For each finding f in F_raw:
    k = <f.host, f.port, f.endpoint, f.cwe_id>
    G[k] = G[k] U {f}
Consolidated Set S_t = {}
For each group in G:
    f_p = MergeFindings(group)
    If f_p.cve_id: f_p.cvss = QueryNVD_CIRCL(f_p.cve_id)
    f_p.owasp, f_p.pci_dss = MapCompliance(f_p)
    f_p.danger = EvaluateRealDanger(f_p)
    f_p.recipes = GenerateFixRecipes(f_p)
    S_t = S_t U {f_p}
If S_{t-1} is provided:
    Delta_NEW = { f in S_t | Hash(f) not in Hash(S_{t-1}) }
    Delta_FIXED = { f in S_{t-1} | Hash(f) not in Hash(S_t) }
    Delta_UNCHANGED = { f in S_t | Hash(f) in Hash(S_{t-1}) }
Return S_t, <Delta_NEW, Delta_FIXED, Delta_UNCHANGED>
```

Plain-English Danger Engine and Executive Posture: The DangerEngine maps technical findings into 4 Real-World Business Threat Categories: (i) Customer & Database Theft (SQLi, directory exposure, credential leaks); (ii) Account Hijacking & Auth Bypass (JWT weaknesses, brute-force, missing cookie flags); (iii) Phishing, XSS & Brand Defacement (CORS wildcards, XSS, open redirects); and (iv) Full Server Takeover & RCE (remote command injection, SSTI). The engine computes an Executive Security Posture Grade (A+ to F, 0–100 score) and provides non-technical explanations of what is broken, what an attacker can achieve, business liability, and estimated fix time.

Multi-Framework Fix Recipes and Dev Tickets: The RecipeEngine generates drop-in code patches for Node.js/Express (parameterized queries, Helmet.js, express-validator), Python/FastAPI/Django (SQLAlchemy ORM, Pydantic schemas, Talisman headers), Nginx server hardening directives, and Cloudflare/AWS WAF virtual patching rules. It also exports 1-click Markdown tickets pre-formatted for Jira, Linear, and GitHub Issues.

Set-Theoretic Delta Differential Auditing: To track regressions between software builds [14], the DeltaAnalyzer computes an identity hash H(f) over each finding. Findings between current scan S_t and baseline S_{t-1} are partitioned into three disjoint sets: Delta_NEW = { f in S_t | H(f) not in H(S_{t-1}) }, Delta_FIXED = { f in S_{t-1} | H(f) not in H(S_t) }, and Delta_UNCHANGED = { f in S_t | H(f) in H(S_{t-1}) }.

D. Autonomous AI Remediation via Model Context Protocol (MCP) and CI/CD Delivery

AutoSecAudit embeds a standard JSON-RPC 2.0 stdio Model Context Protocol (MCP) Server (mcp_server.py), bridging dynamic vulnerability scanning directly into modern AI developer tools (Cursor, Claude Code, Antigravity) [15]. Table II lists the MCP tools exposed for autonomous AI remediation.

TABLE II AUTOSECAUDIT MCP TOOLS FOR AUTONOMOUS AI REMEDIATION

MCP Tool Name	Arguments	Description and Agent Capability
autosec_scan	target_url, profile, crawl	Triggers automated scans; returns executive threat summary, posture grade, and finding counts.
autosec_get_findings	scan_id, severity, lens	Retrieves enriched findings with Plain-English Danger assessments, CVSS, and technical traces.
autosec_get_fix_recipe	finding_id, framework	Returns drop-in code fix patches (nodejs, python, nginx, waf) for automated source file edits.
autosec_verify_fix	target_url, scanner_name, param	Runs targeted single-scanner re-verification to confirm if an AI fix resolved the vulnerability.
autosec_list_scanners	none	Lists all 19 dynamic scanner plugins, attack descriptions, and profile memberships.

For continuous integration pipelines, AutoSecAudit enforces build gating via the --fail-on parameter (Low, Medium, High, Critical). If any finding meets or exceeds threshold theta, the process returns exit code 1 to halt risky releases. Results are delivered via Server-Sent Events (SSE) to a real-time web dashboard and compiled into ReportLab executive PDF reports.

IV. EXPERIMENTAL EVALUATION AND RESULTS

We evaluated AutoSecAudit on an 8-core host (16 GB RAM) running Ubuntu 22.04 LTS against a containerized instance of the OWASP Juice Shop benchmark. Our evaluation answers four research questions: RQ1 (Latency & Concurrency), RQ2 (Accuracy & Alert Noise Reduction), RQ3 (Delta Regression Tracking Fidelity), and RQ4 (System Reliability & Test Suite Coverage).

A. Concurrency and Execution Latency (RQ1)

We measured total audit latency across worker thread counts (k in {1, 2, 4, 8, 16}). As presented in Table III and Fig. 2, increasing concurrency from sequential execution (k=1) to k=8 reduced scan time from 184.2 s to 64.8 s—a 64.8% reduction in latency (2.84x speedup). Beyond 8 threads, performance leveled off due to network socket I/O serialization on the target host.

TABLE III AUDIT LATENCY ACROSS CONCURRENCY CONFIGURATIONS

Worker Threads (k)	Scan Duration (s)	Speedup	CPU Utilization (%)
1 (Sequential)	184.2 s	1.00x	14.2%
2	102.5 s	1.80x	27.8%
4	71.4 s	2.58x	51.3%
8 (Optimal)	**64.8 s**	**2.84x**	**78.4%**

16	66.1 s	2.79x	82.1%

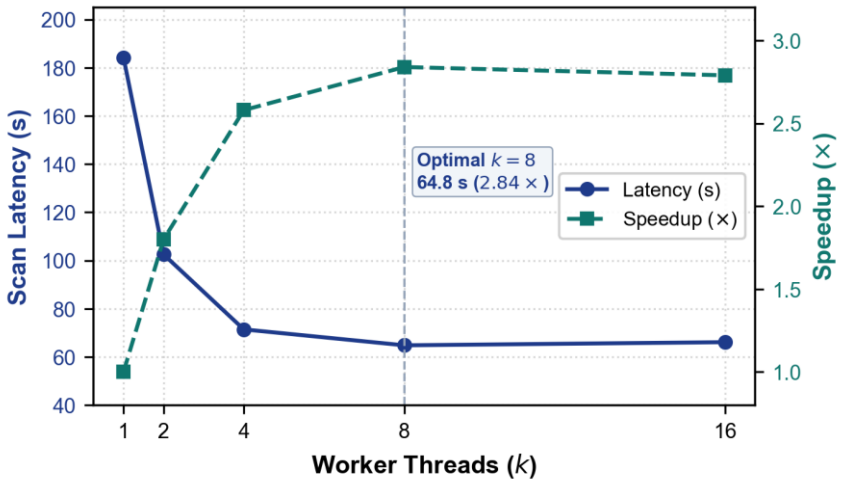


Fig. 2. Audit execution latency and relative speedup across worker thread allocations (k in {1, 2, 4, 8, 16}).

B. Detection Accuracy and Noise Reduction (RQ2)

To evaluate detection accuracy, we audited the benchmark application and cross-referenced findings against ground-truth challenge definitions [10]. Table IV details results across all 19 plugins grouped into major vulnerability classifications.

TABLE IV VULNERABILITY DETECTION AND CORRELATION METRICS ACROSS 19 PLUGINS

Vulnerability Class (Active Plugins)	True Positives (TP)	False Positives (FP)	Raw Alerts	Correlated Findings
Injection (SQLi, CmdInj, SSTI, OpenRedirect)	8	0	14	8
Cross-Site Scripting (XSSScanner, CSRFScanner)	12	1	19	13
Misconfig & Recon (Misconfig, DirBrute, Nikto, Nmap, TLS, Secrets)	15	0	26	15
Access & Traversal (BOLAIdor, Auth, PathTrav, SSRF)	6	0	9	6
API & Tokens (APIAbuse, JWT, CORS)	5	0	8	5
Total / Summary	**46**	**1**	**76**	**47 (-38.2% Noise)**

Across 76 raw tool alerts, the intelligence layer consolidated findings into 47 distinct actionable issues (46 TP + 1 FP), achieving a 38.2% reduction in duplicate alert volume (Fig. 3). Evaluated against 52 verified dynamic challenge vulnerabilities in benchmark scope, AutoSecAudit achieved 97.9% precision (46/47), 88.5% recall (46/52), and an F1-score of 92.9%.

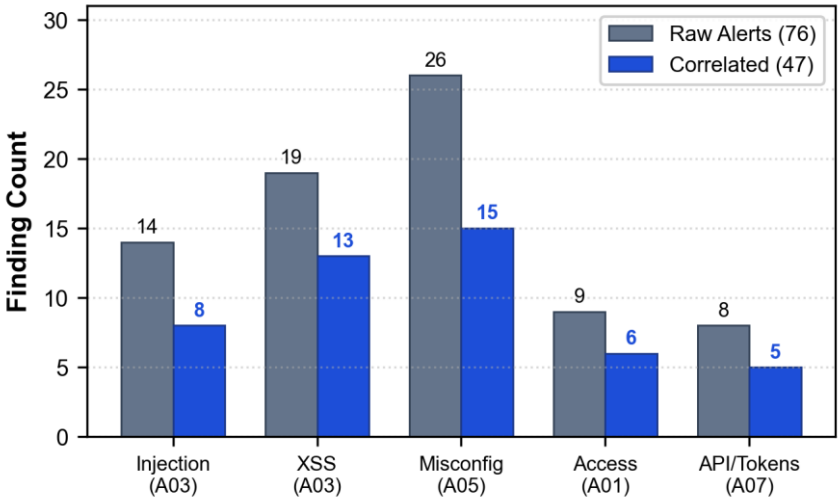


Fig. 3. Raw tool alerts vs. correlated actionable findings across major vulnerability categories (-38.2% alert reduction).

C. Set-Theoretic Delta Regression Tracking (RQ3)

We validated regression tracking across a three-phase synthetic release sequence (Fig. 4): (1) Build v1.0 (Baseline): Initial scan identified 47 correlated vulnerabilities; (2) Build v1.1 (Patch & New): 10 misconfigurations were patched and 3 new API access flaws were introduced. The delta analyzer correctly reported 3 Delta_NEW, 10 Delta_FIXED, and 37 Delta_UNCHANGED issues (47 - 10 + 3 = 40 active issues); and (3) Build v1.2 (Hardened Release): 14 vulnerabilities were resolved. The delta analyzer classified 0 Delta_NEW, 14 Delta_FIXED, and 26 Delta_UNCHANGED issues (40 - 14 = 26 active issues). Across all transitions, the delta engine achieved 100% classification fidelity against release ground truths.

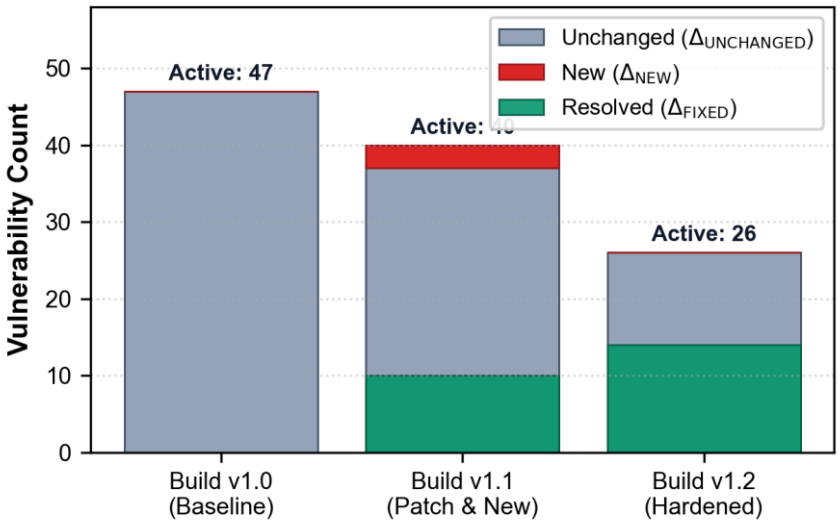


Fig. 4. Vulnerability lifecycle tracking across sequential software builds via set-theoretic delta diffing.

D. System Reliability and Test Suite Coverage (RQ4)

AutoSecAudit maintains high operational stability verified by an automated test suite of 165 unit and integration tests. The test suite exercises all 19 scanner plugins, the Statistical Timing Engine and dual-baseline calibration, deep recursive JSON fuzzing, the Plain-English Danger Engine, multi-framework code fix recipes, the Model Context Protocol (MCP) server tools, and CI/CD exit-code gating.

V. DISCUSSION AND DEPLOYMENT CONSIDERATIONS

Deploying dynamic security scanning into production CI/CD workflows introduces critical engineering considerations: (1) Dynamic Client-Side Single Page Applications (SPAs): While regex extraction uncovers embedded REST API endpoints, complex client-side applications benefit from pairing the crawler with headless browser drivers (e.g., Playwright) for full DOM event execution; (2) Air-Gapped Environments: When scanning within networks without external internet access, AutoSecAudit falls back to an embedded offline CVE/CWE dictionary to prevent

external API timeouts; and (3) Enterprise Authentication: Complex identity provider flows (such as SAML 2.0 or mTLS) require pre-authenticated session tokens injected via environment configuration.

VI. CONCLUSION AND FUTURE WORK

AutoSecAudit provides an extensible, low-latency vulnerability auditing framework and autonomous AI remediation platform for modern DevSecOps pipelines. By orchestrating parallel execution across 19 specialized scanner plugins, the system reduces scan latency by 64.8% (2.84x speedup) over sequential auditing. The post-processing pipeline mitigates developer friction by eliminating 38.2% of duplicate alerts via compound-key deduplication, automatically generating compliance metadata, translating findings into plain-English business danger assessments with executive posture grades, and tracking regression lifecycles with 100% classification fidelity. Furthermore, through standard Model Context Protocol (MCP) integration, AutoSecAudit bridges dynamic security auditing directly into AI coding workflows, closing the loop from vulnerability detection to verified code remediation. Future work will investigate local Large Language Model (LLM) agents for automated patch synthesis and distributed multi-node scanning architectures.

ACKNOWLEDGMENT

This research was conducted as part of the AutoSec Security Engineering Initiative in the Department of Computer Science and Engineering at Don Bosco Institute of Technology, Bangalore. The authors express sincere gratitude to faculty mentor Ms. Ranjitha R, department leadership, and the open-source security engineering community for foundational tools, benchmark platforms, and ongoing support.

REFERENCES

[1] G. F. Lyon, Nmap Network Scanning: The Official Nmap Project Guide to Network Discovery and Vulnerability Scanning. Insecure.Com LLC, 2009.

[2] C. Sullo and D. Lodge, "Nikto Web Server Scanner," Open Source Security Tool, 2023. Available: https://cirt.net/Nikto2

[3] ProjectDiscovery, "Nuclei: Fast and Customizable Vulnerability Scanner Based on Simple YAML DSL," 2024. Available: https://github.com/projectdiscovery/nuclei

[4] OWASP Foundation, "OWASP Zed Attack Proxy (ZAP)," 2024. Available: https://www.zaproxy.org/

[5] OWASP Foundation, "OWASP DefectDojo: Vulnerability Management Orchestration," 2024. Available: https://www.defectdojo.org/

[6] OWASP Top 10 Team, "OWASP Top 10: 2021 The Ten Most Critical Web Application Security Risks," OWASP Foundation, 2021. Available: https://owasp.org/Top10/

[7] PCI Security Standards Council, Payment Card Industry Data Security Standard (PCI-DSS): Requirements and Testing Procedures, Version 4.0, 2022.

[8] National Institute of Standards and Technology (NIST), "National Vulnerability Database (NVD) REST API v2.0," U.S. Department of Commerce, 2023.

[9] M. Myrbakken and R. Colomo-Palacios, "DevSecOps: A Multivocal Literature Review," in Communications in Computer and Information Science, vol. 770, Springer, 2017, pp. 17–29.

[10] B. Bau, F. Bursztein, D. Gupta, and J. C. Mitchell, "State of the Art: Automated Black-Box Web Application Vulnerability Testing," in Proc. 31st IEEE Symposium on Security and Privacy (S&P '10), 2010, pp. 332–345.

[11] A. Doupé, M. Cova, and G. Vigna, "Enemy of the State: A State-Aware Black-Box Web Vulnerability Scanner," in Proc. 21st USENIX Security Symposium (USENIX Security '12), 2012, pp. 523–538.

[12] V. Atlidakis, P. Godefroid, and M. Polishchuk, "RESTler: Stateful REST API Fuzzing," in Proc. 41st IEEE/ACM International Conference on Software Engineering (ICSE '19), 2019, pp. 748–758.

[13] B. Johnson, Y. Song, E. Murphy-Hill, and R. Bowdidge, "Why Don't Software Developers Use Static Analysis Tools to Find Bugs?" in Proc. 35th IEEE/ACM International Conference on Software Engineering (ICSE '13), 2013, pp. 672–681.

[14] C. Vassallo, S. Proksch, H. C. Gall, and M. Di Penta, "Continuous Code Quality: How Developers Manage Technical Debt in CI/CD," IEEE Transactions on Software Engineering (TSE), vol. 46, no. 8, pp. 838–855, 2020.

[15] Anthropic, "Model Context Protocol (MCP) Specification: Open Standard for Seamless AI Application Integration," 2024. Available: https://modelcontextprotocol.io

BIOGRAPHY

The authors are undergraduate research scholars and faculty mentors in the Department of Computer Science and Engineering at Don Bosco Institute of Technology (DBIT), Bangalore, affiliated with Visvesvaraya Technological University (VTU), Belagavi, Karnataka, India. Their research focuses on automated vulnerability assessment, DevSecOps engineering, intelligent security orchestration, and agentic AI remediation systems.